

Prova Técnica - Desenvolvedor .NET Pleno

API REST para gestão de cartões de crédito

1. Contexto

Você foi contratado para construir uma API REST para uma empresa de cartões de crédito. A API deve permitir que um usuário autenticado consulte e gerencie seus próprios cartões. Use o arquivo SQL fornecido como base inicial de dados.

O foco da avaliação é verificar domínio prático de REST, .NET, persistência, validação, segurança básica, tratamento de dados sensíveis, paginação, filtros, atualização parcial e organização de código.

2. Stack esperada

- API em C# com .NET 8 ou superior.
- Persistência usando PostgreSQL. O uso de Entity Framework Core é recomendado, mas não obrigatório.
- Swagger/OpenAPI habilitado para facilitar a execução manual dos endpoints.
- Arquitetura organizada em camadas ou abordagem equivalente, evitando regras de negócio diretamente em controllers.
- README com instruções de execução, decisões técnicas e comandos necessários.

3. Autenticação e rotação de token

Todas as rotas de cartões devem exigir autenticação. A estratégia de autenticação fica a critério do candidato, desde que seja simples de executar localmente e esteja documentada no README.

A autenticação deve identificar o usuário autenticado. Um usuário não pode consultar, criar, alterar ou remover cartões vinculados a outro usuário.

O token ou credencial temporária utilizado na autenticação não deve ser tratado como permanente. A implementação deve considerar validade de **30 minutos**, rejeitar credenciais expiradas e documentar uma estratégia simples de rotação/renovação. A rotação deve substituir o valor anterior por um novo valor, com nova validade de 30 minutos.

Não é necessário implementar cadastro de usuários. A forma de emissão, validação e renovação da autenticação fica a critério do candidato, desde que o fluxo seja demonstrável e não exponha credenciais em logs desnecessários, mensagens de erro ou respostas de listagem.

4. Requisitos funcionais

Implemente os cinco métodos HTTP abaixo para o recurso cartão.

Método	Endpoint sugerido	Comportamento esperado
GET	/api/cards	Listar cartões do usuário autenticado. A listagem deve permitir navegação em blocos fixos de 10 registros, retornar primeiro os cartões criados mais recentemente e aceitar restrição por período de vencimento informado na requisição.
GET	/api/cards/{id}	Consultar um cartão específico do usuário autenticado.
POST	/api/cards	Criar um novo cartão para o usuário autenticado.
PUT	/api/cards/{id}	Atualizar completamente os dados de um cartão existente.
PATCH	/api/cards/{id}	Atualizar parcialmente os dados de um cartão existente.
DELETE	/api/cards/{id}	Remover um cartão. Após a remoção, ele não deve aparecer nas consultas comuns do usuário.

4.1 Listagem de cartões

A listagem deve retornar apenas cartões do usuário autenticado. O retorno deve ser organizado do cartão mais novo para o mais antigo, considerando a data de criação do registro.

Como alguns usuários podem ter muitos cartões, a consulta deve entregar os resultados em páginas de tamanho fixo, com metadados suficientes para o cliente saber em qual página está, quantos registros existem e se há novas páginas disponíveis. O usuário Mariana Alves possui mais de 10 cartões justamente para validar esse comportamento.

Também deve ser possível restringir a listagem por um intervalo de vencimento do cartão. A forma exata dos parâmetros fica a critério do candidato, desde que esteja clara no Swagger/README e que o filtro seja aplicado antes da materialização dos dados.

Formato mínimo esperado para uma resposta paginada:

```
{
  "page": 1,
  "pageSize": 10,
  "totalItems": 12,
  "totalPages": 2,
  "items": [
    {
      "id": "uuid-gerado-pelo-banco",
      "nickname": "Principal",
      "brand": "VISA",
      "maskedNumber": "5321 **** * 5336",
      "expirationDate": "2028-01-31",
      "creditLimit": 12000.00,
      "status": "ACTIVE",
      "createdAt": "2026-06-30T09:10:00Z"
    }
  ]
}
```

4.2 Criação de cartão

O endpoint de criação deve receber os dados necessários do cartão, incluindo o número do cartão e uma senha numérica de uso do cartão. Essas informações devem ser tratadas como críticas: não devem aparecer em respostas comuns da API, logs, mensagens de erro ou listagens.

```
POST /api/cards
Content-Type: application/json

{
  "cardholderName": "MARIANA ALVES",
  "nickname": "Cartão Eventos",
  "brand": "VISA",
  "cardNumber": "5321123412345336",
  "expirationDate": "2029-12-31",
  "creditLimit": 6500.00,
  "pin": "1234"
}
```

Valide campos obrigatórios, formato da data, limite maior ou igual a zero, status permitido e consistência do cartão em relação ao usuário autenticado. Em caso de sucesso, retorne dados suficientes para o cliente identificar o cartão criado sem expor informações sensíveis. O número do cartão deve ser apresentado de forma mascarada, por exemplo: **5321 **** * 5336**.

4.3 Atualização completa - PUT

O PUT deve substituir os dados editáveis do cartão. Defina claramente no README quais campos são editáveis e como campos ausentes serão tratados.

```
PUT /api/cards/{id}
Content-Type: application/json

{
  "cardholderName": "MARIANA ALVES",
  "nickname": "Principal Atualizado",
  "brand": "VISA",
  "cardNumber": "5321123412345336",
  "expirationDate": "2029-01-31",
  "creditLimit": 14000.00,
  "status": "ACTIVE",
  "pin": "9876"
}
```

4.4 Atualização parcial - PATCH

O PATCH deve permitir atualização parcial de campos editáveis sem exigir o envio do objeto completo. A implementação pode usar JSON Patch, Merge Patch ou um DTO próprio de atualização parcial, desde que documentado.

```
PATCH /api/cards/{id}
Content-Type: application/json

{
  "nickname": "Uso diário",
  "creditLimit": 15500.00
}
```

4.5 Remoção - DELETE

O DELETE deve fazer com que o cartão deixe de aparecer nas consultas comuns. A operação precisa preservar consistência, não permitir remoção de cartão de outro usuário e não quebrar rastreabilidade mínima do domínio.

```
DELETE /api/cards/{id}
```

A resposta deve deixar claro o resultado da operação sem expor detalhes internos ou permitir acesso a cartões de outro usuário.

4.6 Consulta exclusiva da senha do cartão

A senha do cartão deve poder ser consultada em um endpoint exclusivo para esse fim. Esse fluxo deve ser separado das consultas comuns de cartão e precisa demonstrar cuidado adicional com autenticação, autorização, exposição de dados sensíveis e registro de logs.

A implementação deve permitir recuperar a senha original quando o acesso for válido, sem armazená-la em texto puro. Caso o modelo inicial fornecido no SQL não seja suficiente para isso, evolua o modelo via migration ou script complementar e documente a decisão técnica no README.

5. Regras de negócio e validações

- 1 A API deve limitar qualquer operação ao usuário autenticado.
- 2 Tokens ou credenciais temporárias devem possuir validade de 30 minutos e uma forma documentada de rotação/renovação.
- 3 A listagem deve trabalhar com blocos fixos de 10 itens. O usuário Mariana Alves possui mais de 10 cartões para validar esse requisito.
- 4 A ordenação da listagem deve sempre priorizar o cartão criado mais recentemente.
- 5 O filtro por vencimento deve aceitar um período de datas definido na requisição e documentado pelo candidato.
- 6 O status do cartão deve aceitar apenas ACTIVE, BLOCKED ou CANCELLED.
- 7 O número do cartão não deve ser retornado integralmente em consultas comuns; use mascaramento no formato equivalente a 5321 **** * 5336.
- 8 CreditLimit não pode ser negativo.
- 9 A senha do cartão deve ser tratada como dado altamente sensível em todo o fluxo.
- 10 A consulta da senha deve existir apenas em endpoint exclusivo e exigir autorização adequada.
- 11 Erros devem ser padronizados e usar status HTTP coerentes.

6. Entregáveis

- Código-fonte da API.
- README com instruções para rodar localmente.
- Script SQL executado ou instrução de migration utilizada.
- Breve explicação das decisões técnicas adotadas.

Observação: o arquivo SQL fornecido é uma base inicial. Caso identifique necessidade de ajustes no modelo para cumprir os requisitos com segurança e rastreabilidade, documente a decisão no README e entregue os scripts/migrations correspondentes.